Norwegian University of Science and Technology (NTNU)

Faculty of Information Technology and Electrical Engineering

Embedded AI

Project

Author: Bukueva Elena
Supervisor: Geir Mathisen
Date: Trondheim, November 3, 2025

Task Description

Goal

Explore Edge AI constrains and deployment tools through the task of Real-Time Fasteners Detection on Raspberry Pi 4&5. The system must run on-device, produce stable detections suitable for downstream sorting and meet latency, memory and environment constraints typical for edge environments.

Operating Context & Constraints

- Hardware: Raspberry Pi 4 (4 GB), Raspberry Pi 5 (8 GB) and Hailo-8 (26 TOPS) accelerator.
- Camera: UI-3590CP-C-HQ Rev.2 - IDS uEye running up to 21 FPS with configurable exposure(e.g., 100 000 µs)
- Environment: Conveyor Belt with chosen speed during inference - 2 cm/s; occasional vibration and lighting drift.

Method

- **Modeling**
  - Defining the set of suitable for Edge AI models;
  - Exploration theirs constraints and performance;
- **Annotation & Dataset**
  - Camera SDK setup and programming;
  - Define annotation format and tools;
  - Perform annotation and prepare suitable for the chosen model dataset;
- **Training**
  - Perform a Transfer Learning Training on a GPU;

  - Defining the pretrained weights based on the task and required annotation quality;

- **Quantization & Compilation**
  - Configuration of a Hailo Compiler for the Hailo-8 AI Accelerator deployment;
  - INT8 PTQ with calibration set; Export .onnx → .har → .hef;

- **Edge Pipeline**
  - Programming of the Camera Real-Time Streaming;
  - Perform Real-Time Inference;
  -

- **Benchmarking**
  - CPU vs Hailo-8 AI comparison;
  - FPS and Detection Metrics measures;
  - Model Performance Comparison;

Deliverables

- Annotated Dataset
- Trained Weights (FP32 + INT8) and compiled `.hef` for YOLOv8 & YOLO11 (-nano and -small)
- Benchmark Report (tables, plots of accuracy, latency, FPS)
- Deployment Guide (Raspberry Pi setup, uEye drivers installation guide, Hailo API configurations, Calibration pack)

Abstract

This project investigates practical Edge-AI deployment for the Real-Time fastener detection on Raspberry Pi 4/5, with and without Hailo-8 AI accelerator. It targets a detection on a conveyer belt with motion $\approx 2cm/s$, using an IDS uEye camera with maximum 21 FPS capability. A lightweight YOLO variants (v8n/v8s/v11n/v11s) are trained via transfer learning on a domain-specific fastener dataset. Four classes of objects are used during the training: machine screws, wood screws, nuts and washers. An additional fifth class is used to identify incomplete parts of the object, when neither a human, not a model can correctly classify an object. The dataset is prepared with the help of Roboflow framework, and it consists of 2131 augmented 960x960 grayscale images. Models are exported to ONNX format and compiled with the Hailo toolchain (ONNX $\rightarrow$ HAR $\rightarrow$ HEF) using INT8 post-training quantization from a calibration set.

A discussion of the annotation format choices, compilation workflow and on-device inference constraints is provided in the literature overview section.

Evaluation performed based on standard detection metrics (IoU, AP, mAP@0.50 and mAPP@[0.5:0.95]) and runtime (latency/FPS). On GPU, all tested YOLO versions archived competitive accuracy on the gathered dataset. While on Raspberry PI 4/5 CPU, real-time throughput is not met, Hailo accelerator shows significantly higher raw inference capacity, having hardware benchmarks $\approx 142FPS$, while Pi 5 pipeline delivers $\approx 1FPS$. However the archieved inference accuracy after the performed quantization is visibly lower than the pure YOLO PyTorch checkpoint inference. The studies on image size and NMS settings, and briefly analyzed quantization effects on small, fine-grained targets are reported.

Contents

Abbreviations

AI	Artificial Intelligence
AABB	Axis-Aligned Bounding Box
AP	Average Precision
API	Application Programming Interface
ARM	Arm architecture (CPU family)
CCW	Counterclockwise
COCO	Common Objects in Context (dataset)
CPU	Central Processing Unit
DL	Deep Learning
ES (in OpenGL ES)	OpenGL for Embedded Systems
FN	False Negative
FP	False Positive
FP32	32-bit floating point
FLOPs	Floating-Point Operations
FPS	Frames Per Second
GFLOPs	Giga Floating-Point Operations
GPU	Graphics Processing Unit
HAR	Hailo Archive (intermediate model package)
HEF	Hailo Executable Format (compiled binary for Hailo device)
HN	Hailo Network (graph JSON inside HAR)
IDS	Imaging Development Systems (uEye camera vendor)
INT8	8-bit integer
IoU	Intersection over Union
L2/L3	Level-2 / Level-3 CPU cache
mAP	mean Average Precision
NMS	Non-Maximum Suppression
NPZ	NumPy Zip archive (weights/tensors)
OBB	Oriented (Rotated) Bounding Box
ONNX	Open Neural Network Exchange
OpenGL ES	OpenGL for Embedded Systems
PTQ	Post-Training Quantization
RAM	Random Access Memory

SDK	Software Development Kit
SoC	System on Chip
TOPS	Tera Operations Per Second
TP	True Positive
uEye	IDS camera/product line
VStream	(Hailo) Virtual Stream (I/O stream abstraction)
Vulkan	Low-level graphics/compute API

1 Introduction

1.1 Background of the Assignment

1.2 Motivation

1.3 Limitations

1.4 Contributions

1.5 Thesis Structure

2 Literature Study

2.1 Datasets and Annonation strategies for Detection Tasks

Object detection models localize objects on an input picture or on a set of video frames. Localization can be performed by different approaches, the most relative to this work are:

- Axis-aligned bounding box (AABB): (x, y, w, h), which is cheap to annotate and fast to predict.
- Oriented/rotated bounding box (OBB): apart from AABB an angle θ of the box is introduced, which is a bit costlier to label.
- Polygon: a list of vertices tracing the object boundary. Provides precise geometry, however is time-consuming to label.
- Pixel mask ("painted area") - most precise 2D shape, ideal for measurement and robotics, heaviest to annotate and train on.

Choosing the right localization strategy should be guided by the structure of the available pretrained weights. Training large detectors from scratch is both resource- and time-intensive. For example, Ultralytics estimates that training YOLOv5 on COCO (118k images, 300 epochs) on a single NVIDIA V100-16GB takes approximately $n/s/m/l/x \approx 1/2/4/6/8$ days, respectively [1]. Collecting and annotating a dataset of that scale is even more time-consuming. Consequently, it is preferable to start from weights pretrained on a dataset whose *labeling granularity* matches the task (e.g., axis-aligned boxes vs. oriented boxes vs. masks). If the required annotation is more detailed than the pretraining labels, an additional annotation effort is needed for transfer learning. Then, if axis-aligned bounding boxes (AABB) are too coarse for small or complex objects, a richer localization (e.g., oriented boxes or instance masks) to meet accuracy requirements would be a better choice.

Mentioned earlier COCO dataset [2] is the common detection benchmark. It is big enough, varied, well-annotated and permissively licensed. The dataset contains photos of 91 objects types with a total of 2.5 million labeled instances in 328k images.

COCO's [2] official detection annotations are axis-aligned boxes only (bbox: [x, y, width, height]). There are no oriented boxes in COCO, so the standard YOLOv8/YOLO11 [3] *detect* checkpoints (trained on COCO 2017) learn AABB by default:


```
{
  "id": 1768,
  "image_id": 397133,
  "category_id": 1,
  "bbox": [73.0, 43.0, 199.0, 305.0],
  "area": 39220.0,
  "iscrowd": 0,
  "segmentation": [
    [78.0, 45.0, 86.0, 45.0, 95.0, 47.0, 101.0, 51.0, ...]
  ]
}
```


Figure 2.1: Example picture from the COCO dataset with AABB annotation [4]

For oriented bounding boxes (OBB), the DOTA dataset [5] is the pioneering benchmark. Ultralytics provides YOLO `*-obb` weights pretrained on DOTA [6].

Other common datasets are:

- Objects365 [7] (600k images, 365 classes): broader coverage than COCO; popular for pretraining backbones/heads before COCO fine-tuning.
- Open Images [8] (millions of images; boxes, masks, relationships): huge scale, but label noise is higher; great for scale, weaker for cleanliness.

None of these datasets, however, include "fasteners" as a class. In order to particularly train model on fasteners, a MVTec screws dataset [9] might be used. This dataset has

oriented-box annotations for screws/nuts (384 images, 4,426 OBBs). However, the labeling.

These are the 13 categories used in the MVTec Screws oriented-box dataset (category_id → name) :

1 → long lag screw
2 → lag wood screw
3 → wood screw
4 → short wood screw
5 → shiny screw
6 → black oxide screw
7 → nut
8 → bolt
9 → large nut
10 → nut
11 → nut
12 → machine screw
13 → short machine screw

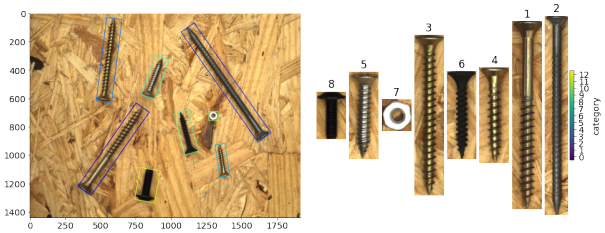


Figure 2.2: Example picture from the MVTec Screws dataset with OBB annotation [10]

It is also possible to convert OBB annotation to AABB annotation with some loss of information. In order to do so:

Let w, h be side lengths and θ the rotation (radians, CCW). The tight axis-aligned box around the rotated rectangle has

$$W_{\mathrm{AABB}} = |\cos\theta|\, w + |\sin\theta|\, h, \tag{2.1}$$

$$H_{\mathrm{AABB}} = |\sin\theta|\, w + |\cos\theta|\, h. \tag{2.2}$$

With the same center (c_x, c_y),

$$x_{\min} = c_x - \frac{W_{\text{AABB}}}{2}, \qquad x_{\max} = c_x + \frac{W_{\text{AABB}}}{2}, \tag{2.3}$$

$$y_{\min} = c_y - \frac{H_{\text{AABB}}}{2}, \qquad y_{\max} = c_y + \frac{H_{\text{AABB}}}{2}. \tag{2.4}$$

If the oriented box is given by four corners (x_i, y_i), the enclosing AABB is

$$x_{\min} = \min_i x_i, \quad x_{\max} = \max_i x_i, \qquad y_{\min} = \min_i y_i, \quad y_{\max} = \max_i y_i. \tag{2.5}$$

COCO bbox uses [$x_{\min}$, $y_{\min}$, width, height], then:

$$\text{bbox}_{\text{COCO}} = \big[\, x_{\min},\ y_{\min},\ x_{\max} - x_{\min},\ y_{\max} - y_{\min} \,\big]. \tag{2.6}$$

YOLO txt uses normalized $(x_{\text{center}}, y_{\text{center}}, w, h)$, then:
Given image size $(W_{\text{img}}, H_{\text{img}})$,

$$x_{\text{center}} = \frac{x_{\min} + x_{\max}}{2\, W_{\text{img}}}, \qquad y_{\text{center}} = \frac{y_{\min} + y_{\max}}{2\, H_{\text{img}}}, \tag{2.7}$$

$$w = \frac{x_{\max} - x_{\min}}{W_{\text{img}}}, \qquad h = \frac{y_{\max} - y_{\min}}{H_{\text{img}}}. \tag{2.8}$$

2.2 The model choice

For Edge AI, the trade-off between computational speed and memory availability is critical. A model must not only deliver sufficiently high accuracy, but also remain compact to fit onto resource-constrained devices such as microcontrollers.

YOLO models family 2.3 presents state-of-the-art lightweight versions of model small and efficient enough to be used on Edge-devices. They show good results for both - the prediction accuracy and inference speed.

Before YOLO [3] the most common detection approaches used two-stage pipelines. The first stage was responsible for generation of the box, the other one for classification. It allowed to archive high accuracy, but kept these models slow. The YOLO series introduced a single-one stage detector - a single convolutional model predicts bounding boxes and class probabilities in one pass over the image. It speed up the inference and allowed YOLO to compete among Real-Time Inference models.

Among 11 versions of YOLO models (see Table [16]), the most important ones are v5, v8, v10 and v11. In some sense, v5, introduced by Ultralytics team, created a founda-

Method	Train	mAP	FPS
Real-Time			
100Hz DPM [11]	2007	16.0	100
30Hz DPM [11]	2007	26.1	30
Fast YOLO	2007+2012	52.7	**155**
YOLO	2007+2012	**63.4**	45
Less Than Real-Time			
Fastest DPM [12]	2007	30.4	15
R-CNN Minus R [13]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN (VGG-16) [15]	2007+2012	73.2	7
Faster R-CNN (ZF) [15]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 2.1: Real-Time Systems on PASCAL VOC 2007. Fast YOLO is the fastest detector and YOLO attains higher mAP while remaining real-time. [3]

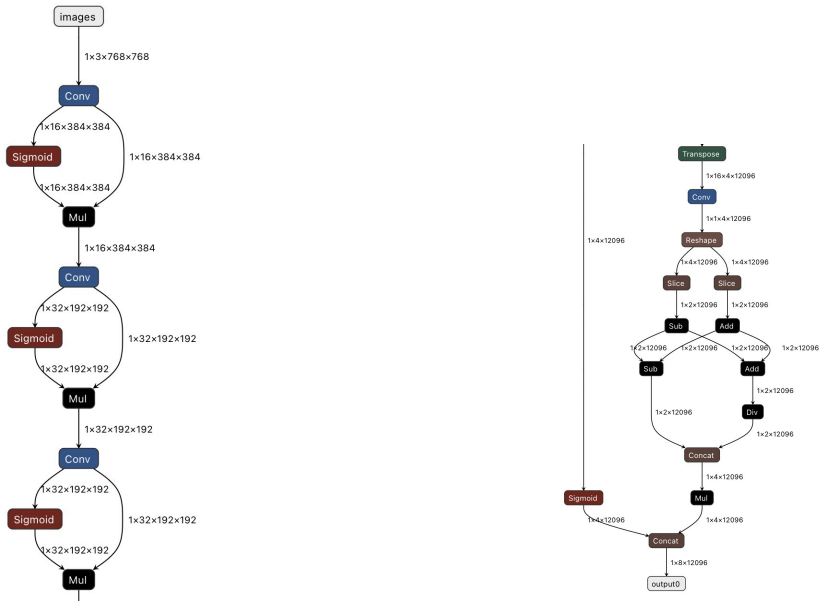
tion for other versions. It came out in PyTorch implementation, which made it easy to deploy, it had multiple sizes (s, m, l, x), so was is easy to adjust to the specific task and constraints. It also became possible to export model to formats for mobile deployment (as ONNX). Version 8 2.3 focused more on improving model performance and flexibility. Ultralytics improved the backbone. The model adopted anchor-free detection: it predicts box coordinates and class probabilities without predefined anchor boxes. It simplified the training process and improved its generalization. Version 11 released in 2024 by a Tsinghua University group introduced an important strategy for the box deduplication. Normally YOLO models need an NMS post-process algorithm to remove duplicate detections of the same object, but version 10 presented NMS-free detection. It uses a dual-label assignment during the training process, so the model predicts only one high-confidence box per object. In version 10 large-kernel convolutions and partial self-attention also were introduced to increase the receptive field without heavy computations. The results were impressive. Yolov10-s model reported 1.8 times faster inference compared to its competitor - RT-DETR model with similar AP, while having 2.8 times fewer parameters (see Table 2.2).

The last published version - YOLO11 - didn't introduce significant changes in the architecture structure and is mostly based on YOLOv8 design. For example, it still used NMS to remove duplicated boxes. Authors were focused on optimising the network and achieving higher accuracy per parameter. It jumps in accuracy and efficiency over YOLOv8 and shows higher mAp with fewer parameters.

There also exist some other successful approaches in real-time detection. One of the earlier mentioned models - RT-DETR - uses a Transformer-based architecture. By us-

Table 2.2: Comparisons of RT-DETR versus YOLOv8 and YOLOv10 models. Latency^f denotes the latency of the forward pass without post-processing.

Model	#Param.(M)	FLOPs(G)	AP^{val} (%)	Latency (ms)	Latency^f (ms)
YOLOv8-N [17]	3.2	8.7	37.3	6.16	1.77
YOLOv10-N [18]	**2.3**	**6.7**	**38.5/39.5**†	**1.84**	**1.79**
YOLOv8-S [17]	11.2	28.6	44.9	7.07	2.33
RT-DETR-R18 [19]	20.0	60.0	46.5	4.58	4.49
YOLOv10-S [18]	**7.2**	**21.6**	**46.3/46.8**†	**2.49**	**2.39**



(a) Start of the graph (b) End of the graph

Figure 2.3: ONNX representation of YOLOv8 layers.

ing transformer decoder, it removes NMS entirely, as was done in YOLOv10 too. It allows RT-DETR model performs excellent on GPU by efficiently utilizing transformers parallelization. However, YOLO models still remain more parameter-efficient and faster on low-power device or CPUs. The other well suitable for the Edge detection model is NanoDet. It targets fast CPU inference and deployment on edge devices. It can be as small as 0.95 million parameters, which is smaller than YOLOv8n and Yolov11n models. Authours report a quantized INT8 NanoDet model having approximately 97 FPS on a mobile CPU. In spite of the speed, the model is notably less accurate, having 34% mAP with YOLOv8n having 37% and YOLOv11n - 39.5%.

Summary of comparison: For edge hardware with limited or no GPU, YOLOv11-n or NanoDet would be preferred due to their small size (with YOLO offering higher accuracy at a bit larger size). With a GPU or NPU, YOLOv11-s or even RT-DETR can be considered.

2.3 Hardware

Among the most popular development platforms is the Raspberry Pi family, which offers a flexible setting for deploying lightweight AI systems. Raspberry Pi 5 used for this work includes:

Compute	
SoC	2.4 GHz quad-core 64-bit Arm Cortex-A76 (crypto ext.); 512 KB L2 per core, 2 MB shared L3
GPU	VideoCore VII; OpenGL ES 3.1, Vulkan 1.3
Memory and Storage	
RAM	8 GB
microSD	64 GB

An additional hardware used during the experiment is a Hailo-8 accelerator offering 26 TOPS inferencing performance respectively.

However, the YOLOv8 and YOLOv11 are both provided by *Ultralytics* as a PyTorch checkout. In order to use it on accelerator, this checkout has to be converted to a suitable format by Hailo Compiler.

2.4 Hailo Compiler

Hailo compiler [20] is needed to load user's models to the Hailo devices. The final result of the compiler is a binary file `.hef`. The standard DataFlow for this conversion consists of the three main steps:

- Translation of the source model to Hailo model;
- Optimization of model parameters;
- Compilation process itself;

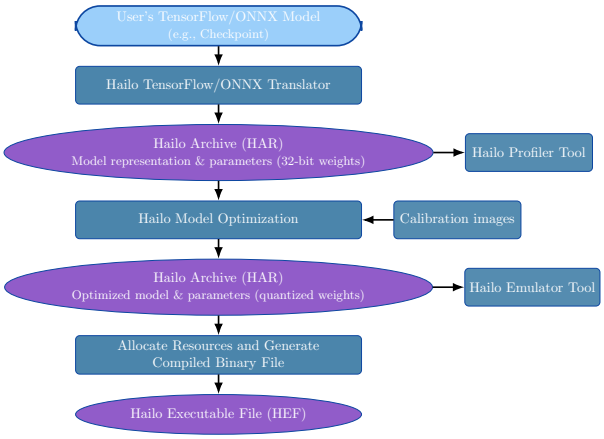


Figure 2.4: Build Process [20]

The first step in the Figure 2.4 is a translation from the user's model to the `.har` representation. The Hailo Translation API parses user's model and produces a `.har` (Hailo ARchive).

- HAR is a zip package that contains:
  - HN— "Hailo Network": a JSON graph of the network (layers, tensor shapes/dtypes, connections, metadata).
  - NPZ—NumPy arrays with the floating-point weights(and sometimes other tensors like batch normalization params).

This step is needed, so the parameters of the model are normalized from the other possible checkpoint configurations (used operation and attributes names, FP32 weights), and the optimizer has a clean input.

The second step is model optimization. The key part of the optmization is the conversion of the model from FP32 to INT8. It is done with an additional run of calibrarion module.

There exist different ways to quantize a model. To my knowledge, Hailo Compiler is using the Linear Quantization:

Key benefits from this approach is its efficiency, support by major DL frameworks and easy implementation [21]. However, the drawback is a visible precision loss as many nearby floating-point values collapse to the same quantized level. Layers with large dynamic range (e.g., softmax) are more sensitive to this method.

Assume an input floating-point value, the linear quantization is

$$q = \text{round}\left(Z + \frac{r}{S}\right).$$

Where

- r: original 32-bit floating-point number
- q: quantized representation (integer)
- S: scale factor (step size)
- Z: zero point (integer), which allows $r = 0$ to be represented exactly by the integer Z

Dequantization is defined by the formula:

$$r \approx (q - Z)\, S.$$

It is also illustrated at the Figure 2.5

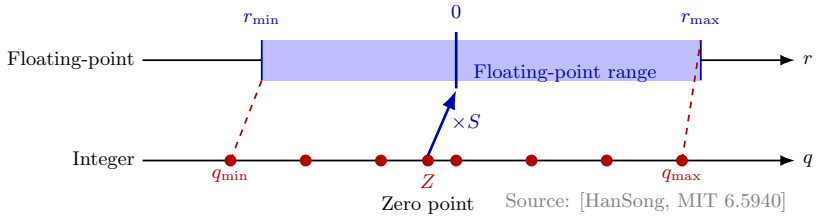


Figure 2.5: Build Process [21]

In order to perform this optimization, the calibration set of raw, unlabeled samples is gathered. It should match a real input distribution (color space, normalization). The compiler uses this set to run a neural network on it and logs per-tensor activation distributions. So, it records min/max values and percentiles. By doing so, an optimizer gets r_{min} and r_{max} values. Which are used to compute q_{min}, q_{max} and Z - point (see Figure 2.5). It is important to do it per tensor as different layers produce values with very different ranges/outliers. Choosing ranges from their actual distributions minimizes quantization error.

The last step is a compilation itself, which the model is compiled to a binary file `.hef`. The compiler is responsible for allocation hardware resources, so the highest possible FPS can be archived. It performs it by mapping neural network layers onto the hardware [22] (compute elements, memory elements, and control elements) and schedule their execution under the chip's resource constraints. If the network is too large, the compiler will partition the network into several contexts, allowing bigger models still run with some performance overhead. The compiler might tile large tensors, split layers into smaller

pieces to fit into memory and to utilize parallelizm. Key strategy of this hardware mapping is to perform as many computation as possible (possibly for difference layers) on the kept on the chip memory tensors. As part of scheduling, compiler also defines the data flow from layer to layer, chaining producers and consumers layer clusters. For example, while the layer i is computed on one side of the chip, another part of the chip might start executing layer i+1 (Illusration: 2.6). This schedule is constructed during the compilation step. Apart from the execution plan, compiler also performs on-chip Non-Maximum Suppression (NMS) fusion for models like YOLO. NMS algorithm performs deduplication of several predicted for the same object boxes. For instance, the screw could be with different probabilities be predicted both as machine and wood screw. So, the post-processing NMS is performed to keep only the most suitable box. Hailo's compiler can inregrate NMS as the final layer of the network, so it runs on the hailo itself and not on the CPU after the inference. It improves the performance not only due to localization of computation, but also due to reduced data transfer - only filtered detections are sent to the Raspberry Pi.

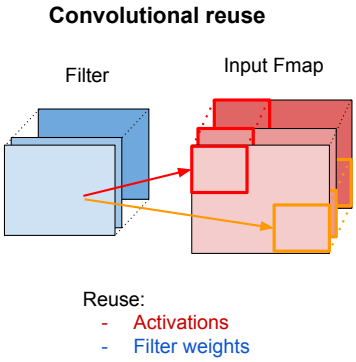


Figure 2.6: Example of the layers reuse: convolutional reuse (Inspired by [23])

The final Hailo Executable File contains:

- Low-level program for the target device (layer schedule, the memory buffer addresses/sizes, instruction sequences etc.)
- Weights and biases
- Metadata and calibration information

2.5 ONNX (Open Neural Network Exchange)

ONNX is an open format developed to unify ML models for a variety of frameworks and compilers by defining a common set of operators and represent models via graph structure.

2.6 Summary of the Literature Study

Table 2.3: Summary of YOLO releases [16]

Release	**Year**	**Tasks**	**Contributions**	**Framework**
YOLO	2015	Object Detection, Basic Classification	Single-stage object detector	Darknet
YOLOv2	2016	Object Detection, Improved Classification	Multi-scale training, dimension clustering	Darknet
YOLOv3	2018	Object Detection, Multi-scale Detection	SPP block, Darknet-53 backbone	Darknet
YOLOv4	2020	Object Detection, Basic Object Tracking	Mish activation, CSPDarknet-53 backbone	Darknet
YOLOv5	2020	Object Detection, Basic Instance Segmentation (via custom modifications)	Anchor-free detection, SWISH activation, PANet	PyTorch
YOLOv6	2022	Object Detection, Instance Segmentation	Self-attention, anchor-free OD	PyTorch
YOLOv7	2022	Object Detection, Object Tracking, Instance Segmentation	Transformers, E-ELAN reparameterisation	PyTorch
YOLOv8	2023	Object Detection, Instance Segmentation, Panoptic Segmentation, Keypoint Estimation	GANs, anchor-free detection	PyTorch
YOLOv9	2024	Object Detection, Instance Segmentation	PGI and GELAN	PyTorch
YOLOv10	2024	Object Detection	Consistent dual assignments for NMS-free training	PyTorch
YOLOv11	2024	Object Detection; (variants for) Instance Segmentation, Pose, OBB	Updated training pipeline and architectures; better accuracy–speed trade-offs; streamlined exports (ONNX/TensorRT) in Ultralytics	PyTorch

3 Theory

Metrics description

Metrics description:

- **Intersection over Union (IoU)**:
  For a predicted box p and ground-truth box g, the IoU formula is:

$$\mathrm{IoU}(p, g) = \frac{|p \cap g|}{|p \cup g|}$$

  IoU plays role of evaluating an object localization.
  It quantifies the overlap between a prediction and ground truth bounding boxes by calculating the ratio of the intersection (overlapping area) to the union (total area covered by both boxes). IoU is a building block for calculating mAP.
  The resulting score is a value between 0 and 1, which quantifies how accurately a model has located an object in an image. A score of 1 represents a perfect match, while a score of 0 indicates no overlap at all.
  The key part of using IoU is to define *"IoU threshold"*. This threshold is a predefined value (e.g., 0.5) that determines whether a prediction is correct. If the IoU score for a predicted box is above this threshold, it is classified as a *"true positive"*. If the score is below, it's a *"false positive"*.
  This threshold directly influences other performance metrics like Precision and Recall.

- **Precision and Recall**:
  Precision defines the proportion of true positives among all positive predictions, measuring the ability to avoid misclassification.

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}},$$

  On the other hand, Recall calculates the proportion of true positives among all actual positives, measuring the model's ability to detect all instances of a class.

$$\mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}},$$

- **Average Precision (AP)**:
  AP computes the area under the precision-recall curve [24], providing a single value

that encapsulates the model's precision and recall performance.

$$\mathrm{AP}_\tau = \int_0^1 p(r;\tau)\,dr,$$

In order to build the precision-recall curve, the score (confidence) threshold (t) is used:

- Every detection has a score $s \in [0, 1]$.
- We sweep t from high $\rightarrow$ low (equivalently sort detections by score desc and add them one-by-one).
- At each t we keep detections with $s \geq t$ and recompute P/R.
- This sweep produces the P–R curve.
- AP is the area under this P–R curve;

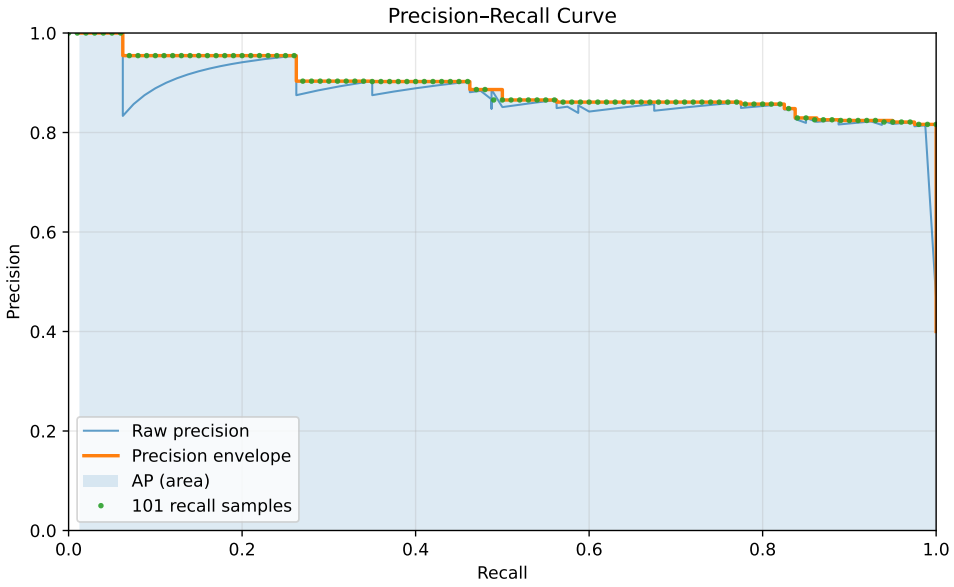


Figure 3.1: Precision–Recall curve.

- **Mean Average Precision (mAP)**:
  Another metric - mAP, extends the concept of AP by calculating the average AP values across multiple object classes.
  This is useful in multi-class object detection scenarios to provide a comprehensive evaluation of the model's performance.

$$\mathrm{mAP}_{0.50} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_{c,\,0.50},$$

$$\mathrm{mAP}_{[0.50:0.95]} = \frac{1}{C}\sum_{c=1}^{C}\frac{1}{10}\sum_{k=0}^{9} \mathrm{AP}_{c,\,(0.50+0.05k)}.$$

The parameter C is the number of classes for which detection and classification are performed.
The subscript 0.5, as well as the set [0.50:0.95], denotes the IoU threshold: predictions with IoU ≥ 0.5 are counted as true positives (TP).

4 Proposed Solution / Design

4.1 Functional Specification

4.2 Architecture / Design Details

5 Implementation

5.1 Key Implementation Details

6 Testing and Results

6.1 Experimental Setup

6.2 Results

Table 6.1: Validation summaries.

(a) YOLO8n [25]

Class	Images	Instances	Box(P)	R	mAP_{50}	$mAP_{50–95}$
all	98	363	0.927	0.951	0.964	0.733
machine_screw	25	123	0.931	0.967	0.989	0.625
miscellaneous	14	16	0.754	0.812	0.845	0.562
nut	16	41	0.988	1.000	0.995	0.777
washer	20	43	1.000	0.991	0.995	0.906
wood_screw	37	140	0.960	0.986	0.994	0.794

Speed per image (GPU): preprocess 4.5 ms; inference 6.0 ms; loss 0.0 ms; postprocess 2.5 ms.

Speed per image (Raspberry Pi 5): preprocess 8.3 ms; inference 1018.4 ms; loss 0.0 ms; postprocess 1.2 ms.

Speed per image (Raspberry Pi 5 + Hailo): Frames: 50, time: 18.5s, avg FPS: 2.71

Model summary (fused): 72 layers, 3,006,623 parameters, 0 gradients, 8.1 GFLOPs

(b) YOLO11s

Class	Images	Instances	Box(P)	R	mAP_{50}	$mAP_{50–95}$
all	98	363	0.939	0.934	0.966	0.746
machine_screw	25	123	0.877	0.983	0.978	0.649
miscellaneous	14	16	0.899	0.688	0.867	0.565
nut	16	41	0.970	1.000	0.995	0.805
washer	20	43	0.973	1.000	0.995	0.909
wood_screw	37	140	0.978	1.000	0.995	0.802

Speed per image (GPU): preprocess 4.5 ms; inference 9.8 ms; loss 0.0 ms; postprocess 3.7 ms.

YOLO11s summary (fused): 100 layers, 9,414,735 parameters, 0 gradients, 21.3 GFLOPs

(c) YOLO11n [26]

Class	Images	Instances	Box(P)	R	mAP_{50}	$mAP_{50–95}$
all	98	363	0.921	0.935	0.964	0.739
machine_screw	25	123	0.844	0.935	0.957	0.611
miscellaneous	14	16	0.922	0.741	0.888	0.608
nut	16	41	0.970	1.000	0.995	0.799
washer	20	43	0.996	1.000	0.995	0.902
wood_screw	37	140	0.873	1.000	0.983	0.773

Speed per image (GPU): preprocess 0.5 ms; inference 5.6 ms; loss 0.0 ms; postprocess 0.9 ms.

100 layers, 2,583,127 parameters, 0 gradients, 6.3 GFLOPs.

Table 6.2: YOLO *nano* (n) variants on COCO at 640×640.

Model	Params (M)	FLOPs (B)	$mAP_{50–95}$ (val)
YOLOv5n	1.9	4.5	28.0
YOLOv8n	3.2	8.7	37.3
YOLO11n	2.6	6.5	39.5

Numbers from Ultralytics model cards: v5 [27], v8 [17], v11 [6].

Table 6.3: YOLO *small* (s) variants on COCO at 640×640.

Model	Params (M)	FLOPs (B)	$mAP_{50–95}$ (val)
YOLOv5s	7.2	16.5	37.4
YOLOv8s	11.2	28.6	44.9
YOLO11s	9.4	21.5	47.0

Numbers from Ultralytics model cards: v5 [27], v8 [17], v11 [6].

Table 6.4: Hailo benchmark results for `yolov8_custom`.

Metric	Mode	Value
FPS	HW-only	142.371
FPS	Streaming	142.370
Latency	HW	10.796 ms

Table 6.5: HEF metadata (`yolov8_custom.hef`).

Field	Value
Architecture	HAILO8
Network group	`yolov8_custom` (Single Context)
Network	`yolov8_custom/yolov8_custom`

Table 6.6: VStream infos.

Direction	Name	Spec
Input	`yolov8_custom/input_layer1`	`UINT8, NHWC(960×960×3)`
Output	`yolov8_custom/yolov8_nms_postprocess`	`FLOAT32, HAILO NMS BY CLASS` (number of classes: 5, max bboxes/class: 100, m

Table 6.7: Post-process (`YOLOV8-Post-Process`) parameters.

Parameter	Value
Op / Name	`YOLOV8 / YOLOV8-Post-Process`
Score threshold	0.001
IoU threshold	0.70
Classes	5
Cross classes	`false`
NMS results order	`BY_CLASS`
Max bboxes per class	100
Image size	960 × 960

7 Discussion

7.1 Discussion Points

8 Conclusion

9 Future Work

- Item one
- Item two
- Item three

Bibliography

[1] Ultralytics, "Yolov5 quickstart tutorial." https://docs.ultralytics.com/yolov5/quickstart_tutorial/. Accessed: 2025-10-29.

[2] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *European Conference on Computer Vision (ECCV)*, Springer, 2014.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] Flickr user, original photographer (via Microsoft COCO), "Coco 2017 validation image id 285 (bear)." http://farm8.staticflickr.com/7434/9138147604_c6225224b8_z.jpg, 2017. Licensed under Creative Commons Attribution 2.0 (CC BY 2.0), http://creativecommons.org/licenses/by/2.0/. Image ID 285 from COCO 2017 validation set; annotations licensed under CC BY 4.0.

[5] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, "Dota: A large-scale dataset for object detection in aerial images," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[6] Ultralytics, "Yolo11: Model card and benchmarks." https://github.com/ultralytics/ultralytics. Accessed: 2025-10-29.

[7] S. Shao, Z. Li, T. Zhang, C. Peng, G. Yu, X. Zhang, J. Li, and J. Sun, "Objects365: A large-scale, high-quality dataset for object detection," in *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 8430–8439, 2019.

[8] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Malloci, A. Kolesnikov, *et al.*, "The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale," *International journal of computer vision*, vol. 128, no. 7, pp. 1956–1981, 2020.

[9] M. Ulrich, P. Follmann, and J.-H. Neudeck, "A comparison of shape-based matching with deep-learning-based object detection," *tm-Technisches Messen*, vol. 86, no. 11, pp. 685–698, 2019.

[10] M. S. GmbH, "Mvtec screws dataset (version 1.0)." `https://www.mvtec.com/research`, 2020. Released with HALCON 19.05. Licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0), `https://creativecommons.org/licenses/by-nc-sa/4.0/`. Contains 384 images of 13 screw and nut categories with oriented bounding box annotations.

[11] M. A. Sadeghi and D. Forsyth, "30hz object detection with dpm v5," in *Computer Vision – ECCV 2014*, pp. 65–79, Springer, 2014.

[12] J. Yan, Z. Lei, L. Wen, and S. Z. Li, "The fastest deformable part model for object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2497–2504, IEEE, 2014.

[13] K. Lenc and A. Vedaldi, "R-cnn minus r," *arXiv preprint arXiv:1506.06981*, 2015.

[14] R. B. Girshick, "Fast r-cnn," *arXiv preprint arXiv:1504.08083*, 2015.

[15] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *arXiv preprint arXiv:1506.01497*, 2015.

[16] R. Khanam and M. Hussain, "Yolov11: An overview of the key architectural enhancements," *arXiv preprint arXiv:2410.17725*, 2024.

[17] Ultralytics, "Explore ultralytics yolov8," 2023.

[18] C.-Y. Wang *et al.*, "YOLOv10: Real-time end-to-end object detection," *arXiv preprint*, 2024. arXiv:2405.14458.

[19] J. Lv *et al.*, "RT-DETR: Real-time detection transformer," *arXiv preprint*, 2023. arXiv:2304.08069.

[20] Hailo Technologies Ltd., *Hailo Dataflow Compiler User Guide*. User guide.

[21] S. Han, "6.5940: Tinyml and efficient deep learning computing." `https://www.youtube.com/@efficientml-ai`, 2024. EfficientML.ai lecture series.

[22] D.-I. B. Hammoud, "Embedded machine learning: Chapter 12—ai hardware accelerators." Lecture 12, Department of Electrical and Computer Engineering, RPTU Kaiserslautern–Landau, 7 2025.

[23] J. Emer, V. Sze, and Y.-H. Chen, "Dnn accelerator architectures." ISCA Tutorial, 2019. Slide deck.

[24] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[25] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8," 2023.

[26] G. Jocher and J. Qiu, “Ultralytics yolo11,” 2024.

[27] Ultralytics, “Yolov5: Model card and benchmarks.” https://github.com/ultralytics/yolov5. Accessed: 2025-10-29.

A Text (Appendix A)

B Text (Appendix B)